

1. Introduction

TinyURL is a URL shortening service that converts long web addresses into short, easy-to-share links. Instead of sending a lengthy URL, users can share a compact version that redirects anyone who clicks it to the original webpage. Services like TinyURL and Bitly are commonly used on social media, emails, and messaging platforms because they make links cleaner and easier to manage.

The goal of this system design is to create a TinyURL service that is fast, reliable, scalable, and capable of handling a large number of users and requests.

Example

Original URL

https://github.com/RampalSiyag/SD_24bcs11753_Rampal

Short URL

<https://tinyurl.com/mrjsv93w>

2. Functional Requirements

The TinyURL system should provide the following features:

- Users should be able to enter a long URL and receive a shortened URL.
- Every shortened URL should be unique.
- When a user opens the short URL, they should automatically be redirected to the original website.
- The system should allow URLs to expire after a specified time if required.
- Information about each shortened URL should be stored safely in the database.
- The system should maintain basic analytics such as the number of clicks.
- Registered users should be able to view and manage their shortened URLs.
- Users should also be able to delete URLs that they no longer need.

3. Non-Functional Requirements

Apart from the main features, the system should also satisfy the following quality requirements:

Availability: The service should remain available almost all the time, with an uptime close to 99.99%.

Scalability: The system should be able to support millions of URLs and handle increasing traffic by adding more servers.

Performance: Users should be redirected to the original website within a fraction of a second (preferably under 100 milliseconds).

Reliability: URL data should not be lost even if one of the servers fails.

Durability: Once a URL is stored, it should remain safely stored unless the user deletes it or it expires.

Security: The service should use HTTPS, validate URLs, and protect against misuse through techniques such as rate limiting.

Maintainability: The architecture should be modular so that new features can be added easily in the future.

4. Assumptions

To estimate the system capacity, the following assumptions are made:

- Around 100 million new URLs are created every day.
- Users access shortened URLs much more frequently than they create them.
- The read-to-write ratio is assumed to be 100:1.
- The average length of a long URL is approximately 500 bytes.
- Additional metadata requires around 100 bytes.

5. Back-of-the-Envelope Estimation

URL Creation

Assuming 100 million URLs are created every day,

Write Requests per Second

$100,000,000 \div 86,400 \approx \mathbf{1,157 \text{ requests per second}}$

For simplicity, this can be rounded to **1,200 write requests per second**.

URL Redirection

Since users mainly access existing URLs, we assume a read-to-write ratio of 100:1.

This gives approximately:

10 billion redirects per day

Read Requests per Second

$10,000,000,000 \div 86,400 \approx \mathbf{115,740 \text{ requests per second}}$

This can be rounded to **120,000 read requests per second**.

Storage Estimation

Average storage required for one URL:

- Long URL: 500 Bytes
- Short URL: 10 Bytes
- Metadata: 100 Bytes

Total storage per record $\approx \mathbf{610 \text{ Bytes}}$

Daily Storage:

$100 \text{ million} \times 610 \text{ Bytes} \approx \mathbf{61 \text{ GB}}$

Yearly Storage:

$61 \text{ GB} \times 365 \approx \mathbf{22 \text{ TB}}$

Cache Requirement

Since a small number of URLs usually receive most of the traffic, storing popular URLs in Redis cache will reduce database load. A cache size of around **200 GB** is assumed.

6. System Components

The TinyURL system consists of the following components:

User: Sends requests to create or access short URLs.

Load Balancer: Distributes incoming requests across multiple application servers.

Application Servers: Handle URL creation, redirection, validation, and communication with other components.

Redis Cache: Stores frequently accessed URLs to reduce database queries and improve response time.

URL Database: Permanently stores all URL mappings.

Message Queue: Receives analytics events without slowing down user requests.

Analytics Worker: Processes events from the queue and updates analytics.

Analytics Database: Stores click statistics for reporting purposes.

7. Request Flow

URL Shortening

1. The user submits a long URL.
2. The request reaches the Load Balancer.
3. The Load Balancer forwards the request to an Application Server.
4. The server generates a unique short code using Base62 encoding.
5. The URL mapping is stored in the database.
6. The shortened URL is returned to the user.

URL Redirection

1. The user opens the short URL.
2. The request reaches the Load Balancer.
3. The Application Server checks the Redis Cache.
4. If the URL is found, the user is redirected immediately.
5. If not, the server retrieves the original URL from the database and updates the cache.
6. A click event is sent to the Message Queue.
7. The Analytics Worker records the event in the Analytics Database.
8. Finally, the user is redirected to the original website.